



Programação II

Programação de Computadores II

2025/2026 - João Rebelo - ISTEC - P.O.O. - Introdução



Programação II

Programação de Computadores II

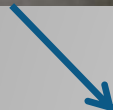
P00 - Introdução

O Conceito “Object Oriented”

- Forma de Resolver Problemas
- Enfoque nos dados e métodos/propriedades associados em vez de ser uma sequência lógica de instruções
- Programa O.O. → Visto como uma coleção de “máquinas” que executam processos e que trocam informação com outras “máquinas” no sistema
- Três princípios fundamentais:
 - Encapsulamento
 - Herança
 - Polimorfismo
- Um objeto é uma instância de uma classe

O Conceito Tradicional vs. O.O.

- A nossa relação com os objetos do dia-a-dia
- Humanos relacionam-se com os objetos de duas formas:
 - Fazendo explicitamente tudo, quando o objeto só “é” e não “sabe”
 - Dando ordens aos objetos quando eles “são” e “sabem”



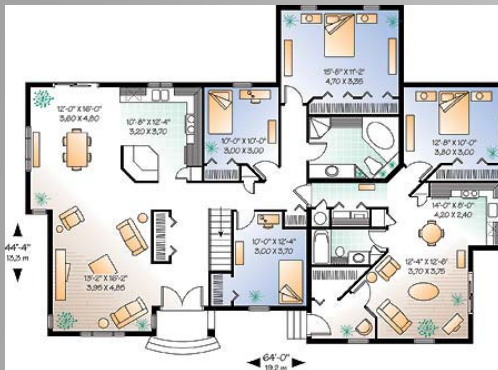
Diferença entre ter que fazer tudo, ou dizer ao objeto para fazer...

O Conceito Tradicional vs. O.O.

- A versão tradicional:
 - Acesso livre a todas as características do objeto (conteúdo)
 - Preocupação em ter que seguir o “algoritmo” à risca
 - Todo o procedimento mais suscetível a erros
 - Um processo simples, pode ter uma escrita e leitura bastante complexa → legibilidade do processo como um todo
- A versão O.O.
 - O acesso está condicionado ao “saber” do objeto (só faço o que ele me permite fazer)
 - Normalmente, utilizam-se as características do objeto através da utilização de “funções” do objeto. Ex: TV, Carro, Cafeteira
 - O “Know-How” está escondido (encapsulado) dentro do objeto
 - Utilizar o objeto, resume-se a utilizar os “botões”, “manípulos” ou “alavancas” do mesmo → Dar ordens ao objeto

O Conceito de “Classe” = “Cápsula”

- “Plano” para construir objetos
- Encapsula:
 - Atributos → o que ela representa ou é
 - Métodos → o que ela saberá fazer
- Classes e Objetos ? Qual a diferença ?



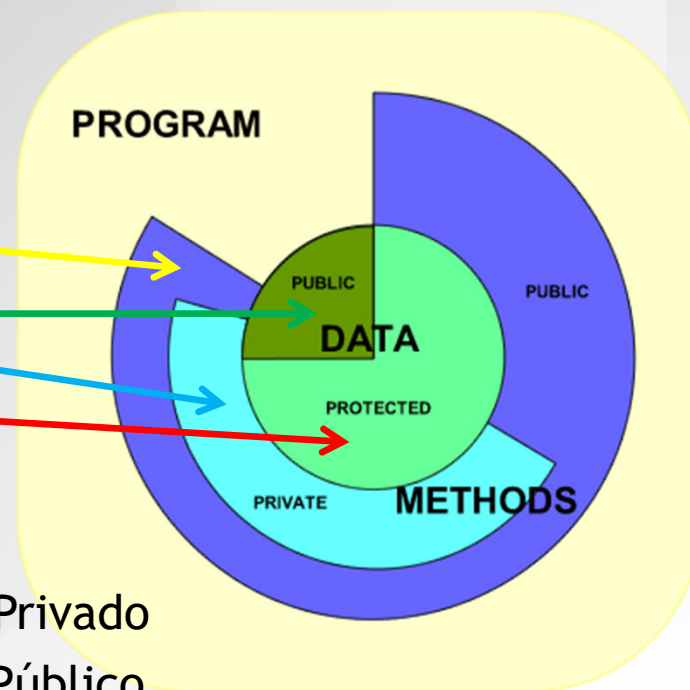
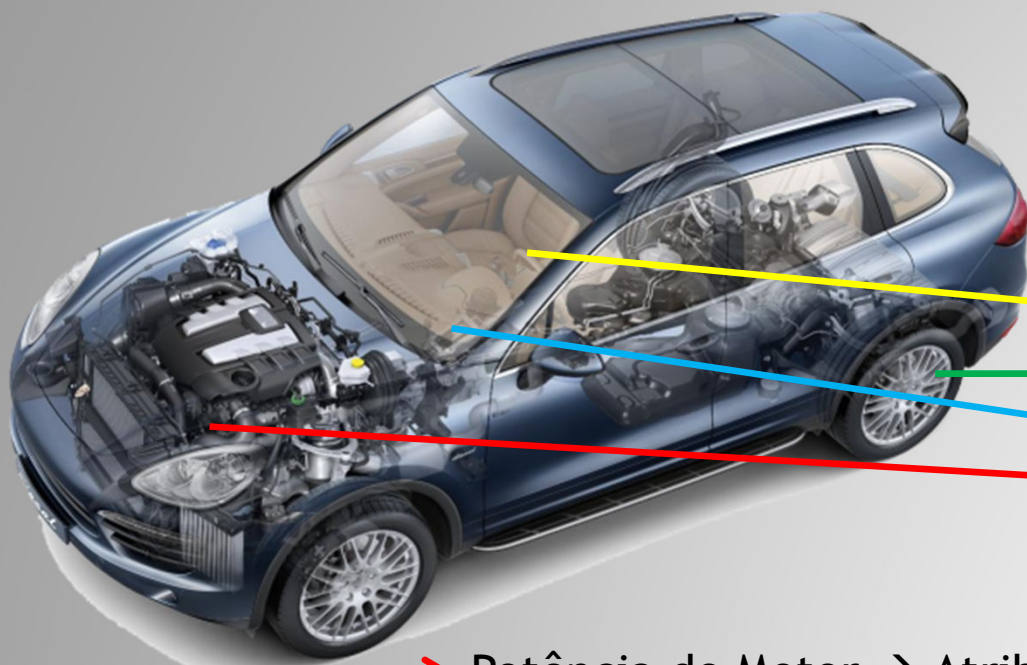
Planta = Modelo → Casa real = Objeto criado a partir do Modelo

CLASSE CASA

OBJETO CASA

O Conceito de Classe - Cont.

- Class → “máquina” que tem características e sabe fazer coisas
- Vista como uma caixa que esconde a complexidade e expõe apenas a forma como deve ser utilizada



- Potência do Motor → Atributo Privado
- Pressão dos Pneus → Atributo Público
- Volante → Método Público
- Aviso Combustível → Método Privado

A Herança

- Possibilidade de criar Classes a partir de outras Classes
 - As novas classes herdam:
 - Atributos
 - Métodos
- Classes Base ou SuperClasses → Servem de infraestrutura para classes mais complexas
- Classes Derivadas ou SubClasses → Classes que herdam membros (características e métodos) de classes base.
- EX:
 - Classe Pessoa → Superclasse
 - Classe Médico → Herda da classe Pessoa
 - Classe Paciente → Herda da classe Pessoa

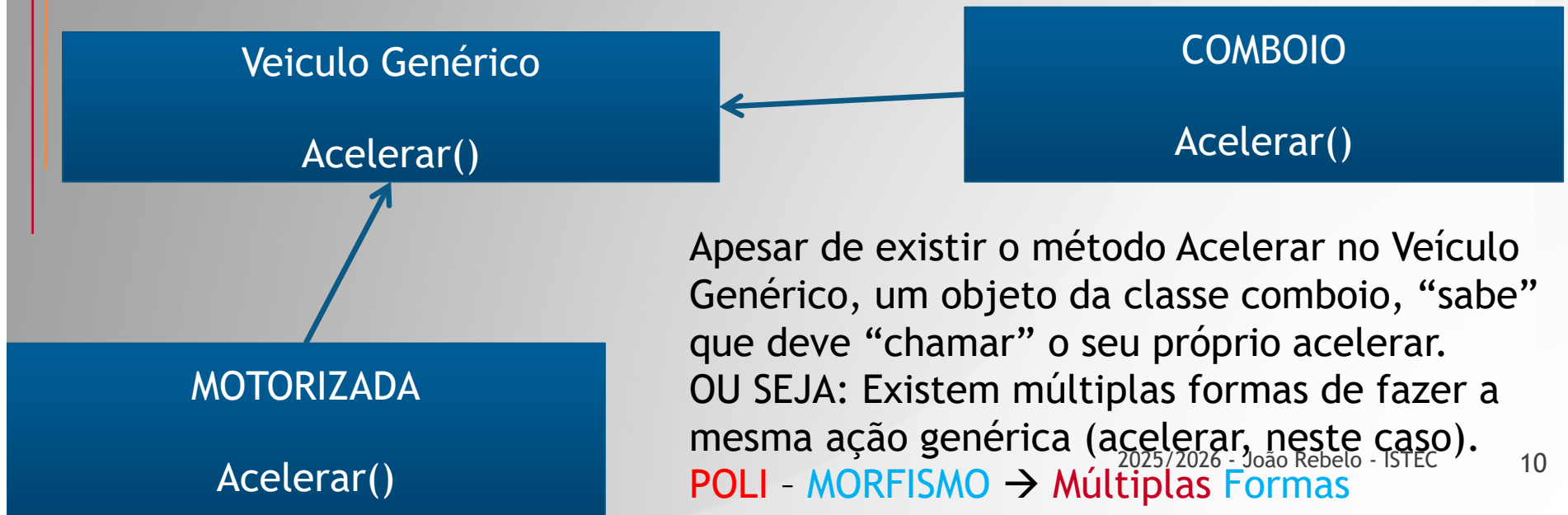
O Polimorfismo

- Do grego *Poli* = muitas, *morphos* = formas
- Múltiplas Formas de fazer a mesma coisa / conceito



O Polimorfismo

- Acelerar é uma funcionalidade comum às três classes
- ACELERAR um veículo é um conceito genérico
- Acelerar uma moto envolve um conjunto de atividades distintas das atividades necessárias para acelerar um comboio.
- No entanto o conceito ACELERAR é comum aos dois tipos de veículo



O Polimorfismo

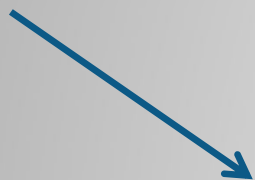
- RESUMINDO:
- Polimorfismo só pode ser considerado num contexto de herança
- Cada objeto sabe que deve invocar a sua própria forma de fazer a funcionalidade, mesmo existindo uma funcionalidade com o mesmo nome na classe base (exemplo: **acelerar**).

Evolução da “Programação”

- No princípio, era a programação **FUNCIONAL** “monobloco” com princípio, meio e fim. Inexistência de estrutura
- Sobressaem dois problemas: a repetição de dados conceitualmente iguais (por exemplo, duas pessoas) e a repetição do mesmo código em partes distintas do programa
 - Cria-se o conceito de Modularidade → blocos com a mesma função ou repetidos em diferentes locais do programa, podiam ser fundidos numa subrotina, que mais tarde evoluiu para função ou procedimento
 - Cria-se o conceito de vetor → conjunto de entidades do mesmo tipo (exemplo, inteiros), agregada no mesmo nome (ex: int v[50])
- Continua a existir o problema da “liberdade” dos dados → em qualquer ponto do programa, é possível modificar de forma descontrolada, o valor de qualquer variável.
 - Cria-se o conceito de estrutura de dados → agrupar atributos relacionados com uma entidade (ex: produto), dentro de uma **caixa (struct)**

Evolução da “Programação”

- Ainda persiste o problema de poder utilizar “funções” impróprias para determinadas entidades (ex: poder-se-ia tentar gravar uma estrutura de dados do tipo pessoa, num ficheiro do tipo cliente).
- Problema: Como limitar uma estrutura de dados a utilizar apenas funções/procedimentos apropriados?
- Resposta: da mesma forma que foi resolvido o problema da “liberdade” dos dados → agrupar os dados de uma entidade numa cápsula, e nesta cápsula guardar também o que a entidade pode fazer (a cápsula É e SABE)



CLASSE

Um exemplo prático:

- Vamos considerar para o efeito, que precisamos criar e mostrar duas características de uma pessoa (das possíveis centenas de cada uma)
- **Introdução:** A forma e método de programar evoluiu desde o simples programa corrido (pouco eficiente e pouco escalável) até às abordagens mais atuais orientadas a objetos, eficientes e escaláveis. Podemos considerar os seguintes estágios evolutivos:
 - Programa “flat”
 - Vetorização
 - Modularização
 - Encapsulação de Informação
 - Encapsulação de Informação e Ações -> orientação a objetos

Bloco “FLAT” - ler 2 pessoas

```
static void Main(string[] args) {  
    string nome1 = "";  
    string nome2 = "";  
    int anoNascimento1 = 0;  
    int anoNascimento2 = 0;  
    int anoAtual = DateTime.Now.Year;  
    //-  
    Console.Write("Digite Nome Pessoa 1:");  
    nome1 = Console.ReadLine();  
    Console.Write("Digite Ano Nascimento da Pessoa 1:");  
    anoNascimento1 = int.Parse(Console.ReadLine());  
    Console.Write("Digite Nome Pessoa 2:");  
    nome2 = Console.ReadLine();  
    Console.Write("Digite Ano Nascimento da Pessoa 2:");  
    anoNascimento2 = int.Parse(Console.ReadLine());  
    //-  
    Console.WriteLine("Pessoa 1:" + nome1 + " nasceu em " + anoNascimento1 + " e tem " + (anoAtual - anoNascimento1) + " anos ");  
    Console.WriteLine("Pessoa 2:" + nome2 + " nasceu em " + anoNascimento2 + " e tem " + (anoAtual - anoNascimento2) + " anos ");  
    Console.ReadLine();  
}
```

- **Problemas da abordagem:**

- Número FIXO e LIMITADO de pessoas
- Possibilidade de confusão entre os dados da pessoa 1 e da pessoa 2
- Completamente desestruturado e gigantesco para muitas pessoas
- Sujeito a erros de inserção de dados

Bloco “FLAT” - com deteção de erros

```
static void Main(string[] args) {
    string nome1 = "";
    string nome2 = "";
    int anoNascimento1 = 0;
    int anoNascimento2 = 0;
    int anoAtual = DateTime.Now.Year;
    //-
    Console.Write("Digite Nome Pessoa 1:");
    nome1 = Console.ReadLine();
    Console.Write("Digite Ano Nascimento da Pessoa 1:");
    try {
        anoNascimento1 = int.Parse(Console.ReadLine());
    }
    catch {
        anoNascimento1 = 0;
    }
    Console.Write("Digite Nome Pessoa 2:");
    nome2 = Console.ReadLine();
    Console.Write("Digite Ano Nascimento da Pessoa 2:");
    try {
        anoNascimento2 = int.Parse(Console.ReadLine());
    }
    catch {
        anoNascimento2 = 0;
    }
    //-
    Console.WriteLine("Pessoa 1:" + nome1 + " nasceu em " + anoNascimento1 + " e tem " + (anoAtual - anoNascimento1) + " anos ");
    Console.WriteLine("Pessoa 2:" + nome2 + " nasceu em " + anoNascimento2 + " e tem " + (anoAtual - anoNascimento2) + " anos ");
    Console.ReadLine();
}
```

- Problemas da abordagem:

- Idênticos ao anterior

Evolução - a vetorização

```
static void Main(string[] args) {
    string[] nome = new string[2];
    int[] anoNascimento = new int[2];
    int anoAtual = DateTime.Now.Year;
    //-
    Console.Write("Digite Nome Pessoa 1:");
    nome[0] = Console.ReadLine();
    Console.Write("Digite Ano Nascimento da Pessoa 1:");
    try {
        anoNascimento[0] = int.Parse(Console.ReadLine());
    }
    catch {
        anoNascimento[0] = 0;
    }
    Console.Write("Digite Nome Pessoa 2:");
    nome[1] = Console.ReadLine();
    Console.Write("Digite Ano Nascimento da Pessoa 2:");
    try {
        anoNascimento[1] = int.Parse(Console.ReadLine());
    }
    catch {
        anoNascimento[1] = 0;
    }
    //-
    Console.WriteLine("Pessoa 1:" + nome[0] + " nasceu em " + anoNascimento[0] + " e tem " + (anoAtual - anoNascimento[0]) + " anos ");
    Console.WriteLine("Pessoa 2:" + nome[1] + " nasceu em " + anoNascimento[1] + " e tem " + (anoAtual - anoNascimento[1]) + " anos ");
    Console.ReadLine();
}
```

- Problemas da abordagem:
 - Apesar do problema das variáveis estar resolvido, temos repetição de código
- Vantagens: número de utilizadores já variável, mas programa ainda não

Evolução - vectorização a sério

```
static void Main(string[] args) {
    int nrPessoas = 0;
    Console.Write("Indique o número de pessoas:");
    try { nrPessoas = int.Parse(Console.ReadLine()); }
    catch { nrPessoas = 0; }
    //
    string[] nome = new string[nrPessoas];
    int[] anoNascimento = new int[nrPessoas];
    int anoAtual = DateTime.Now.Year;
    //- Leitura de Dados para as n Pessoas indicadas
    int Contador = 0;
    while (Contador < nrPessoas) {
        Console.Write("Digite Nome Pessoa " + Contador + ":");
        nome[Contador] = Console.ReadLine();
        Console.Write("Digite Ano Nascimento da Pessoa " + Contador + ":");
        try { anoNascimento[Contador] = int.Parse(Console.ReadLine()); }
        catch { anoNascimento[Contador] = 0; }
        Contador++;
    }
    //- Escrita de informação para as n Pessoas indicadas
    Contador = 0;
    while (Contador < nrPessoas) {
        Console.WriteLine("Pessoa " + Contador + ":" + nome[Contador] + " nasceu em " + anoNascimento[Contador]
            + " e tem " + (anoAtual - anoNascimento[Contador]) + " anos ");
        Contador++;
    }
    Console.ReadLine();
}
```

- Problemas da abordagem:
 - Vários (ex: Código “amontoadado” no programa principal, não reutilizável).

• MAS → Resolvidos os problemas da escalabilidade

Evolução - Ponto Situação

- Conseguimos criar um programa estruturado que permite a leitura de qualquer número de pessoas.
- Continua no entanto, a haver a possibilidade (indesejável) de poder mexer inadvertidamente na informação:
 - Por exemplo, não há qualquer restrição a fazermos...
`nome[0] = "Maria" ou anoNascimento[1] = 20;`
... em qualquer parte do nosso programa
E ISTO NÃO DEVERIA ACONTECER!
- Além disso:
 - Pode ser necessário ter que utilizar novamente as rotinas de inserção e de impressão das pessoas noutros pontos de um programa mais complexo... como fazer?

Evolução - Modularização

- Como convencionar os “módulos” ou “caixas negras” ou “funções/procedimentos” ?
 - Se o código vai ser utilizado várias vezes → modulariza-se
 - Se o código tem um objetivo muito específico → modulariza-se
- Modularizar → Dividir um grande problema num conjunto de pequenas soluções
- Candidatos à modularização no nosso exemplo:
 - A leitura de uma pessoa
 - A escrita de uma pessoa
 - O pedido de numero de pessoas a ler...

Evolução - Modularização 1

```
static string[] nome;  
static int[] anoNascimento;  
static int nrPessoas = 0;  
static int anoAtual = 0;
```

Os dados devem estar fora de qualquer dos módulos/funções, para poderem ser acedidos por qualquer um/uma

0 references

```
static void Main(string[] args) {  
    nrPessoas = 0;  
    Console.WriteLine("Indique o número de pessoas:");  
    try { nrPessoas = int.Parse(Console.ReadLine()); }  
    catch { nrPessoas = 0; }  
    //  
    nome = new string[nrPessoas];  
    anoNascimento = new int[nrPessoas];  
    anoAtual = DateTime.Now.Year;  
    //- Leitura de Dados para as n Pessoas indicadas  
    Ler();  
    //- Escrita de informação para as n Pessoas indicadas  
    Escrever();  
    Console.ReadLine();  
}
```

```
public static void Ler() {  
    int Contador = 0;  
    while (Contador < nrPessoas) {  
        Console.WriteLine("Digite Nome Pessoa " + Contador + ":");  
        nome[Contador] = Console.ReadLine();  
        Console.WriteLine("Digite Ano Nascimento da Pessoa " + Contador + ":");  
        try { anoNascimento[Contador] = int.Parse(Console.ReadLine()); }  
        catch { anoNascimento[Contador] = 0; }  
        Contador++;  
    }  
}
```

```
public static void Escrever() {  
    int Contador = 0;  
    while (Contador < nrPessoas) {  
        Console.WriteLine("Pessoa " + Contador + ":" + nome[Contador] + " nasceu em " + anoNascimento[Contador]  
            + " e tem " + (anoAtual - anoNascimento[Contador]) + " anos");  
        Contador++;  
    }  
}
```

- **Problemas da abordagem:**

- Ainda há uma parte muito específica do programa, no módulo principal (?)

- Vamos resolver o problema ? Ver programa seguinte...

Evolução - Modularização 2

```
static void Main(string[] args) {
```

```
    nrPessoas = QuantasPessoas();
```

```
    nome = new string[nrPessoas];
```

```
    anoNascimento = new int[nrPessoas];
```

```
    anoAtual = DateTime.Now.Year;
```

```
    //- Leitura de Dados para as n Pessoas indicadas
```

```
    Ler();
```

```
    //- Escrita de informação para as n Pessoas indicadas
```

```
    Escrever();
```

```
    Console.ReadLine();
```

```
}
```

```
public static int QuantasPessoas() {
```

```
    int pessoasDevolvidas = 0;
```

```
    Console.WriteLine("Indique o número de pessoas:");
```

```
    try { pessoasDevolvidas = int.Parse(Console.ReadLine()); }
```

```
    catch { pessoasDevolvidas = 0; }
```

```
    return pessoasDevolvidas;
```

```
}
```

```
public static void Ler() {
```

```
    int Contador = 0;
```

```
    while (Contador < nrPessoas) {
```

```
        Console.WriteLine("Digite Nome Pessoa " + Contador + ":");
```

```
        nome[Contador] = Console.ReadLine();
```

```
        Console.WriteLine("Digite Ano Nascimento da Pessoa " + Contador + ":");
```

```
        try { anoNascimento[Contador] = int.Parse(Console.ReadLine()); }
```

```
        catch { anoNascimento[Contador] = 0; }
```

```
        Contador++;
```

```
    }
```

```
}
```

```
public static void Escrever() {
```

```
    int Contador = 0;
```

```
    while (Contador < nrPessoas) {
```

```
        Console.WriteLine("Pessoa " + Contador + ":" + nome[Contador] + " nasceu em " + anoNascimento[Contador] + " e tem " + (anoAtual - anoNascimento[Contador]) + " anos ");
```

```
        Contador++;
```

```
    }
```

```
}
```

- **Problemas da abordagem:**

- Apesar do programa estar modularizado, continua a ser possível manipular livremente a informação das pessoas...

- Vamos encapsular ??? Como ???

Evolução - Encapsular

Características de uma pessoa

Um contentor de pessoas (só a declaração)

A inicialização do contentor de pessoas

```
class Pessoa {  
    public string nome;  
    public int anoNascimento;  
}
```

```
static Pessoa[] vPessoas;  
static int nrPessoas = 0;  
static int anoAtual = 0;
```

References

```
static void Main(string[] args) {
```

```
    nrPessoas = QuantasPessoas();  
    vPessoas = new Pessoa[nrPessoas];  
    anoAtual = DateTime.Now.Year;  
    //- Leitura de Dades para as n Pessoas indicadas  
    Ler();  
    //- Escrita de informação para as n Pessoas indicada  
    Escrever();  
  
    Console.ReadLine();  
}
```

```
public static void Ler() {  
    int Contador = 0;  
    while (Contador < nrPessoas) {  
        vPessoas[Contador] = new Pessoa();  
        Console.Write("Digite Nome Pessoa " + Contador + ":");  
        vPessoas[Contador].nome = Console.ReadLine();  
        Console.Write("Digite Ano Nascimento da Pessoa " + Contador + ":");  
        try { vPessoas[Contador].anoNascimento = int.Parse(Console.ReadLine()); }  
        catch { vPessoas[Contador].anoNascimento = 0; }  
        Contador++;  
    }  
}
```

```
public static void Escrever() {  
    int Contador = 0;  
    while (Contador < nrPessoas) {  
        Console.WriteLine("Pessoa " + Contador + ":" + vPessoas[Contador].nome + " nasceu em " + vPessoas[Contador].anoNascimento  
            + " e tem " + (anoAtual - vPessoas[Contador].anoNascimento) + " anos ");  
        Contador++;  
    }  
}
```

- **Problemas da abordagem:**

- Nenhum... cada pessoa tem a sua própria informação encapsulada...

- Bom, na realidade, ainda há um problema: Uma boa pessoa, deveria saber fazer coisas, tais como, ler a sua própria informação e escrevê-la. Como?

2025/2026 - João Rebelo - ISTE

Evolução - Novo Ponto Situação

- Conseguimos:
 - Modularizar
 - Tornar o programa eficiente
 - Tornar o programa escalável
- Resta resolver um problema:
 - Continua a ser possível utilizar os procedimentos Ler e Escrever, de forma livre, pois são procedimentos genéricos.
 - No entanto, só deveriam ser invocados para lerem e escreverem pessoas... Como resolvíamos?
- Colocávamos os procedimentos, dentro da classe **Pessoa**. Desta forma, a classe passava a:
 - Ser (o que caracteriza a entidade)
 - Saber (o que a entidade é capaz de fazer)

Evolução - Um objeto

```
class Pessoa {  
    public string nome;  
    public int anoNascimento;  
  
    2 references  
    public void Ler() {  
        Console.Write("Digite Nome Pessoa:");  
        nome = Console.ReadLine();  
        Console.Write("Digite Ano Nascimento da Pessoa:");  
        try { anoNascimento = int.Parse(Console.ReadLine()); }  
        catch { anoNascimento = 0; }  
    }  
  
    2 references  
    public void Escrever() {  
        int anoAtual = DateTime.Now.Year;  
        Console.WriteLine("Pessoa " + nome + " nasceu em " + anoNascimento  
            + " e tem " + (anoAtual - anoNascimento) + " anos ");  
    }  
}
```

} Características de uma pessoa. O que ela É!

Métodos de uma
pessoa. O que ela
SABE!

Neste caso:
Ler
Escrever

```
static void Main(string[] args) {  
    Pessoa P1 = new Pessoa();  
    Pessoa P2 = new Pessoa();  
  
    P1.Ler();  
    P2.Ler();  
  
    P1.Escrever();  
    P2.Escrever();  
  
    Console.ReadLine();  
}
```

← Criar duas Pessoas (2 objetos da classe Pessoa)

← Manipular duas Pessoas

- Problemas da abordagem:
- Nenhum...

Exercício:

- Criar em C# uma classe do tipo conta que guarde:
 - Nome, NIF e Saldo da Conta
 - Permita Levantar dinheiro se saldo for suficiente
 - Permita Depositar dinheiro
 - Permita Consultar o estado da conta em qualquer momento
- Exemplo de Utilização:
 - `Conta c = new Conta();`
 - `c.Ler();` //Criar a conta com saldo 0
 - `c.Depositar(500);` //O Saldo deve aumentar 500
 - `c.Levantar(1000);` //Deve dar erro de saldo insuficiente.
 - `c.Levantar(400);` //Deve diminuir o saldo em 400
 - `c.Consultar();` //Deve mostrar os dados da conta
- Submeter no Moodle o .cs criado

Conclusão:

- Ao Criar Cápsulas (structs ou classes) estamos a agrupar:
 - O que a cápsula É → Atributos
 - O que a cápsula SABE fazer → Métodos
- Tendo em conta que:
 - Todos os atributos (nota: TODOS) devem ser privados, isto é, não acessíveis diretamente.
 - Todos os atributos devem ser apenas manipulados por métodos (as funcionalidades das cápsulas que mexem diretamente com os atributos)
- Nestas condições, estamos a implementar a primeira grande características das linguagens orientadas a objetos

■ Cenas dos próximos capítulos:

- Manipularmos todos os atributos de uma entidade, apenas através de métodos, pode ser moroso, complexo e trabalhoso (imagine-se uma entidade cliente com 50 atributos distintos)
- Por outro lado, é expressamente proibido acedermos diretamente aos atributos.
- Então a solução é:

Criar PROPRIEDADES

- Ou o mesmo é dizer:

FORMAS PÚBLICAS E CONTROLADAS DE ACEDERMOS A ATRIBUTOS



Dúvidas e Sugestões

- jrebelo@my.istec.pt

(João Rebelo)